

PANDA serial PG2

使用说明



Version 4.0

免责声明:

森云智能保留在没有任何事先通知的情况下编辑/修改本文件的权力。

目录

一、Windows 使用说明	1
1.1 设备连接与启动	1
1.1.1 安装软件工具	1
1.1.2 连接设备	2
1.1.3 点亮相机	2
1.2 功能说明	4
1.2.1 录像功能	4
1.2.2 拍照功能	5
1.2.3 相机固件读取和更新	5
1.2.4 读写寄存器	8
1.2.5 设置触发	10
1.2.6 读取电流	11
1.2.7 相机内参读取	11
1.2.8 PANDA 固件更新	12
1.2.9 ISP Tuning	16
1.2.10 读取镜头温度	18
二、Linux 使用说明	19
2.1 软件下载	19
2.2 使用方法	19
2.2.1 打开上位机	19
2.2.2 加载配置点亮相机	20
2.2.3 读写寄存器	25

2.2.4 开关控制	26
2.3 Linux 系统脚本点亮方式	27
三、版本记录	30
附录	31
一、UYVY 转 YUYV python 示例	31
二、YUYV 转 UYVY python 示例	33

一、Windows 使用说明

1.1 设备连接与启动

1.1.1 安装软件工具

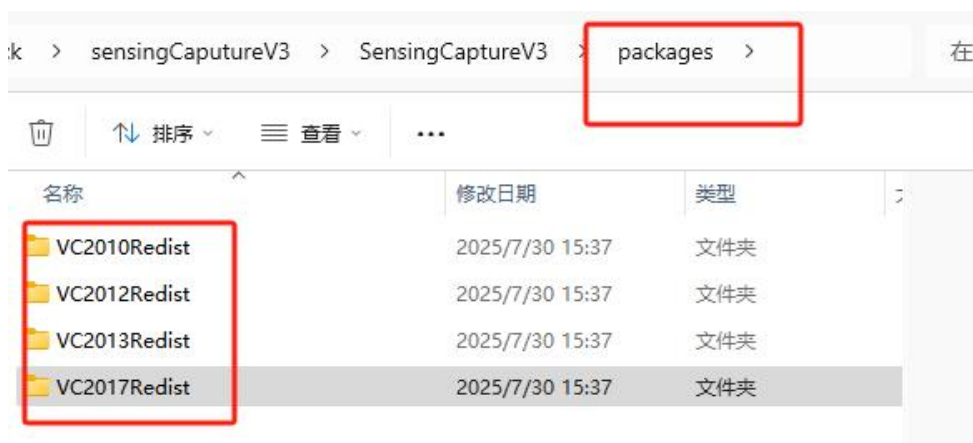
下载链接：<https://github.com/SENSING-Technology/SensingCaptureV3>

拷贝 SensingCaptureV3 安装包到 PC，解压，找到 SensingCaptureV3.exe，双击鼠标左键即可打开，打开后界面如图 1-1 所示。上方和左侧为控制版面，中间为显示区域。



图 1-1

如果首次使用，出现打开时提示缺少一些库文件，可以尝试软件目录下面的 packages 文件夹里面的 VC201XRedist 程序去安装。



1.1.2 连接设备

按照图 1-2 连接设备线束，同轴线接相机，接 1 个相机时，需要接在 CAM1 口的线。

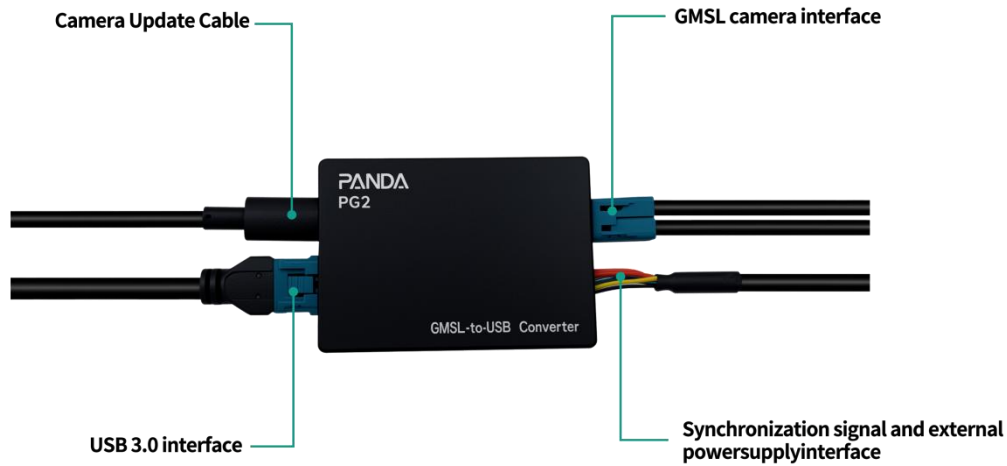


图 1-2

1.1.3 点亮相机

1、打开 SensingCaptureV3

在设备列表、设备信号与控制栏中均可以看到当前摄像头的信息。

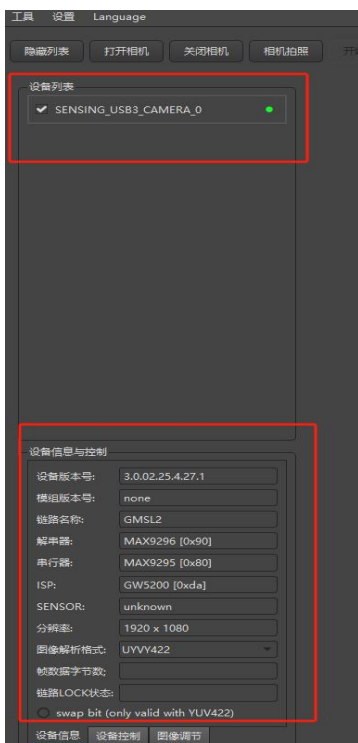


图 1-3

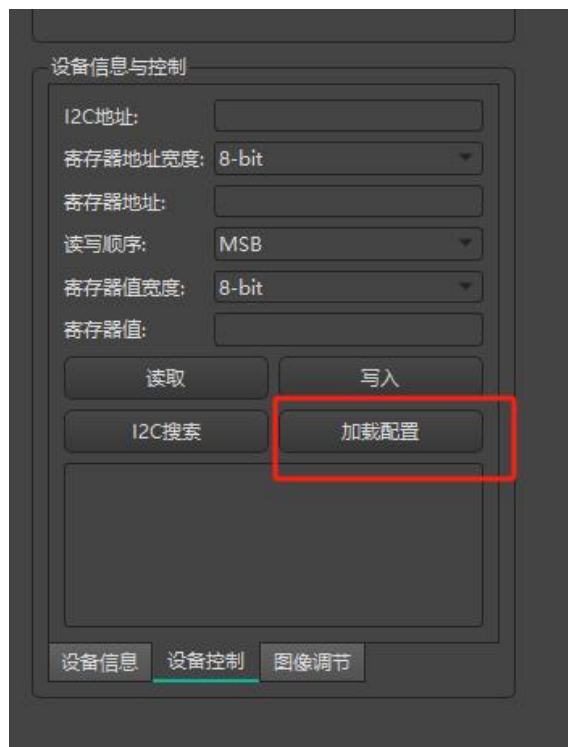


图 1-4

2、加载配置

在设备信息与控制界面中，点击“设备控制”，选择“加载配置”，如图 1-4 所示，系统弹出选择配置文件的提示框。加载配置文件有 2 种方法

方法一：

在配置文件栏，点击小三角，在下拉框中选择相机对应型号的配置文件，如图 1-5

方法二：

在模糊搜索，输入配置文件名称的部分信息，根据配置文件的命名规则：产品型号_分辨率@帧率_图像格式，输入部分信息，如 ISX031，如图 1-6。

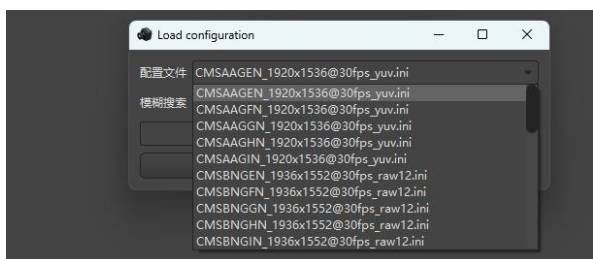


图 1-5

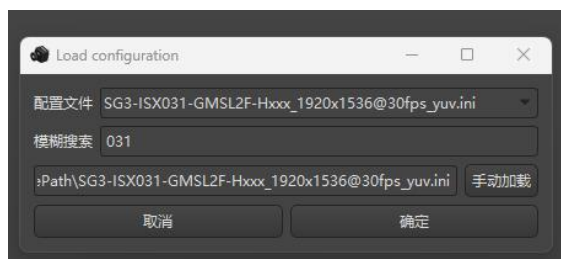


图 1-6

选择好配置文件后，点击“确认”按钮，系统会依据该配置文件的详细信息启动加载流程，待配置文件成功加载，系统将呈现如图 1-7 所示的信息。

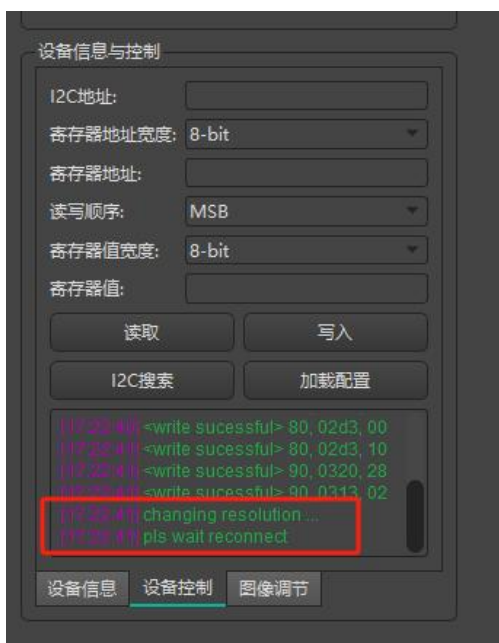


图 1-7

3、打开相机

点击界面左上角，“打开相机”，软件界面中部将呈现摄像头拍摄的图像，如图 1-8.

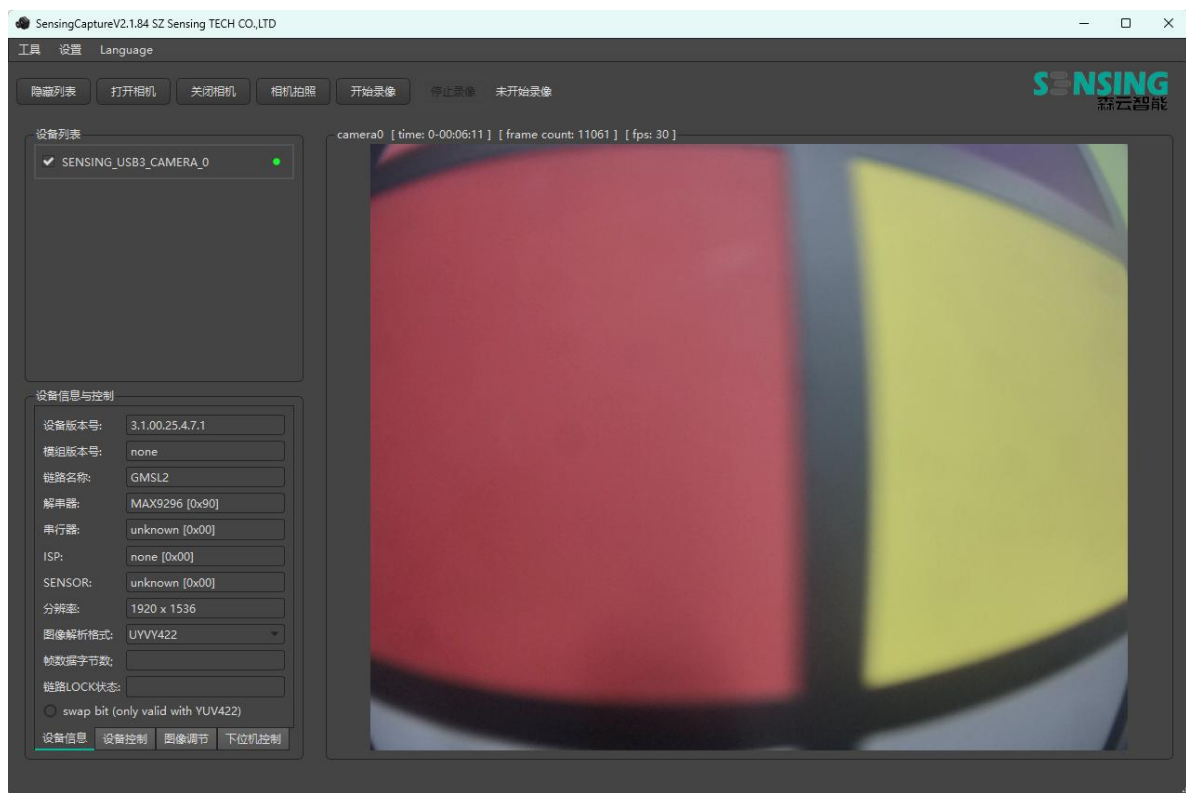


图 1-8

1.2 功能说明

1.2.1 录像功能

1、点击开始录像

打开相机，界面显示如图 1-9，点击“开始录像”，系统会弹出选择保存视频路径的窗口，根据个人的需求，建立合适的保存路径，此次界面会出现“录像中”提示，表明录像正在进行中。录制的视频，文件名为时间，保存的文件类型为 AVI (Audio Video Interleave)，扩展名.avi；录制视频的格式为 Motion-JPEG (MJPEG)。

2、停止录像

点击“停止录像”，界面显示如图 1-10，即可停止录制，在建立的保存路径下，可以

找到录制的视频。

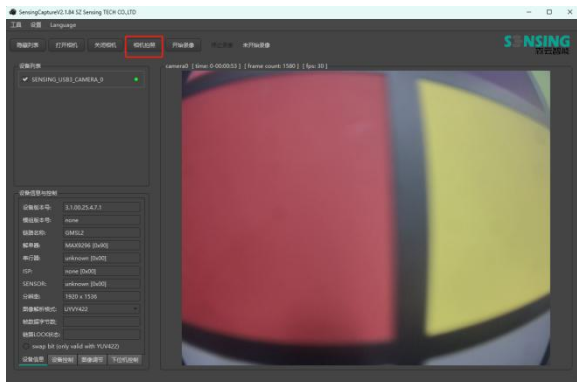


图 1-9

名称	修改日期	类型
bearer	2025/3/14 15:50	文件夹
configs	2025/3/14 15:50	文件夹
iconengines	2025/3/14 15:50	文件夹
imageformats	2025/3/14 15:50	文件夹
pictures	2025/3/14 16:56	文件夹
platforms	2025/3/14 15:50	文件夹
profilePath	2025/3/14 15:50	文件夹
styles	2025/3/14 15:50	文件夹
tmp	2025/5/27 17:12	文件夹
tools	2025/3/27 17:53	文件夹
translations	2025/3/14 15:50	文件夹
update	2025/3/17 13:32	文件夹
WriteIn_Log	2025/4/28 19:44	文件夹

图 1-10

1.2.2 拍照功能

打开相机，点击“相机拍照”，界面显示如图 1-11，即可拍照，文件存在软件安装目录下的 pictures 目录，如图 1-12。

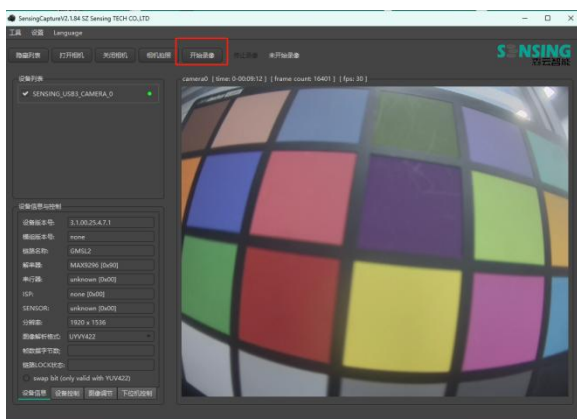


图 1-11

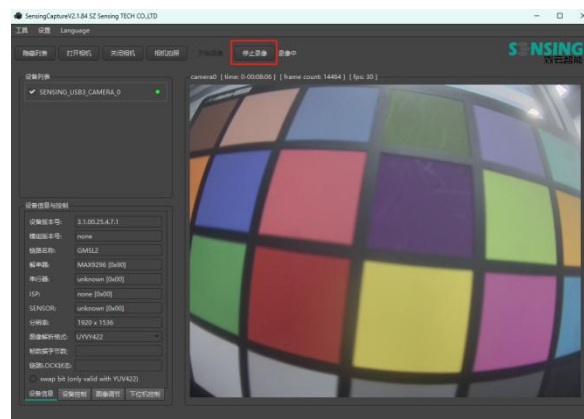


图 1-12

1.2.3 相机固件读取和更新

1、确定线束全部连接

相机升级固件需要确认相机、USB 数据线、Type C 线束（Camera Update Cable）完成连接，如图 1-2 所示。

2、选择 ISP 型号加载固件

通过顶部菜单，点击工具-相机固件升级，根据需要升级的相机 sensor 或 ISP 型号，选择正确型号，如本次要读取的相机型号是 SG3-ISX031C-GMSL2F-H190XA，可以在模

糊搜索框里面输入 031 快速定位到指定型号，选择型号后点击读取版本号可以读取，

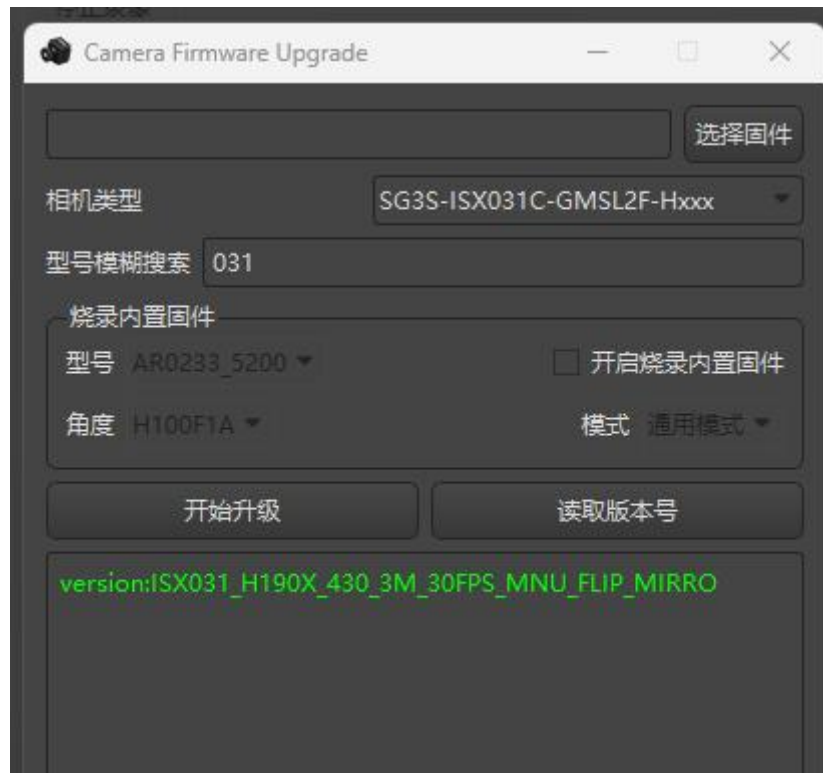


图 1-13

3、更新固件

- (1) 在相机类型中选择模组型号，可以通过模糊搜索快速定位。
- (2) 点击按钮“选择固件”，在文件夹中选择需要更新的固件
- (3) 单击左键开始升级按钮。

具体操作参考图 1-14，固件升级完成后，点击读取版本号，若版本已经升级为新版本，则固件升级成功，如图 1-15。

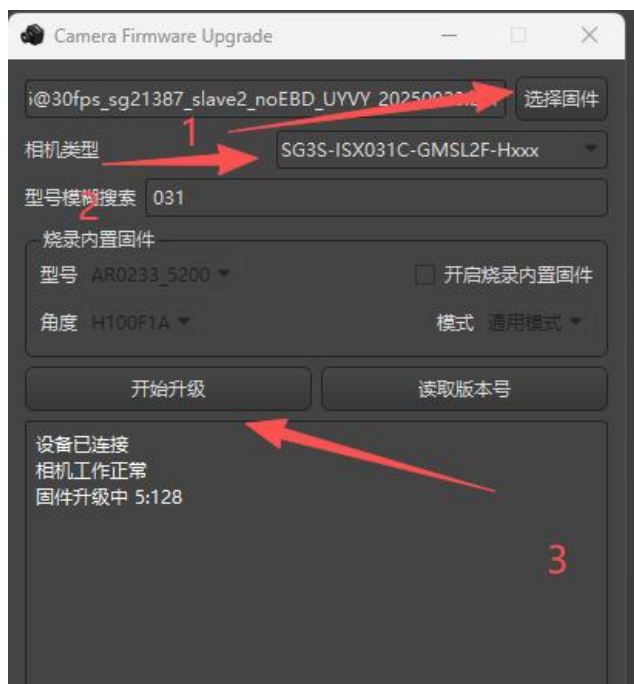


图 1-14

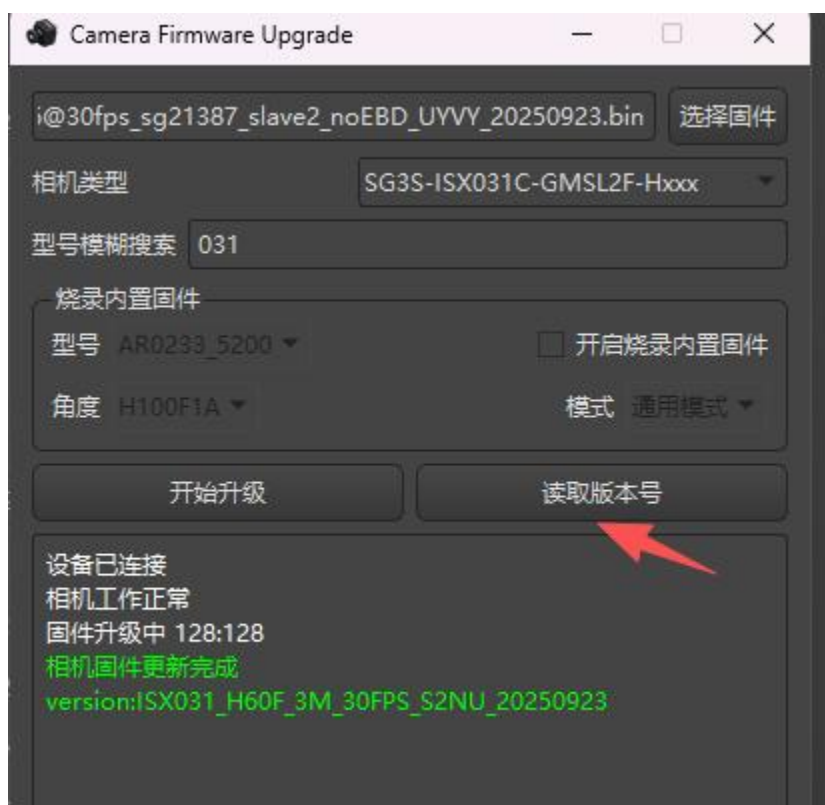


图 1-15

4、切换模式

在软件包里面内置部分固件，可以根据具体使用场景选择指定固件。如下图 1-16 选择，模组

为 SG8-AR0820C-G300-H120YA，选择对应型号、角度和模式点击烧录即可。



图 1-16

1.2.4 读写寄存器

在设备信息与控制界面进行操作，以 SG3S-ISX031C-GMSL2F-H190XA 为例：

1、填写 I2C 地址

根据产品规格书，硬件配置页，查看该产品的 I2C 地址，例如串行器 96717F 的地址默认 8 位地址为 0x80，需要在 I2C 地址框内写入 0x80。地址为 8 位 I2C 地址。

2 、选择寄存器宽度

根据需要选择对应的寄存器宽度，目前支持 8bit 和 16bit 两种。96717F 的寄存器宽度，为 16bit，可以通过产品规格书，硬件配置页查看或 96717F 的 DATASHEET。

3、填写寄存器地址

根据想要读写的寄存器地址，输入本框中，例如，触发信号的寄存器地址为 0X02D3，输入 02D3。

确定读写顺序：在读写顺序下拉菜单中，选择所需的顺序，可选择高位字节优先或低位字节优先。MSB 表示高位字节优先，LSB 表示低位字节优先。本次选择 MSB。

4、选择寄存器值宽度

在寄存器值宽度选项中，选择 8 位或 16 位作为寄存器值的宽度,96717F 为 8 位。

5、输入并操作寄存器值

在寄存器值栏中，输入要读写的值。这里将 0X02D3 写入 0x10，实现将 96717F 的 GPIO 7 作为输出，并拉高。

6、读取

点击读取按钮，若操作成功，将显示该值，如图 1-17 所示，绿色 <read sucessful> 80, 02d3, 83，表示读取成功；红色 <read fail> 80, 02d3, err 则表示读取失败。

7、写入

点击写入按钮，将该值发送至寄存器，如图 1-17 所示，绿色 <write sucessful> 80, 02d3, 10 表示写入成功，红色 <write fail> 80 ,02d3,10 表示写入失败。



图 1-17 读写寄存器

1.2.5 设置触发

在主页面左上角菜单栏中点击工具->触发信号，打开触发信号的控制窗口，打开界面如图 1-18。模组默认为外触发，加载软件内置 slave 的配置，可以将外触发信号给到触发口的 P2 口，触发信号就会透传给模组。触发接口如图 1-19 所示。触发电平支持 1.8V 或 3.3V。



图 1-18



图 1-19

1.2.6 读取电流

将相机接在 PANDA 上，通过读取电流的方式可以估算当前相机的功耗。如图 1-20 所示打开软件在左上角菜单选择 工具->通道控制打开读电流界面，打开界面后如图 1-21，点击获取电流，即可显示对应端口上的电流。如果不用外部供电，使用 USB 端口内部供电，POC 电压约为 11.8V，按照图 1-21 读出电流，计数功率约为 $11.8 \times 0.097 = 1.145W$ 。



图 1-20

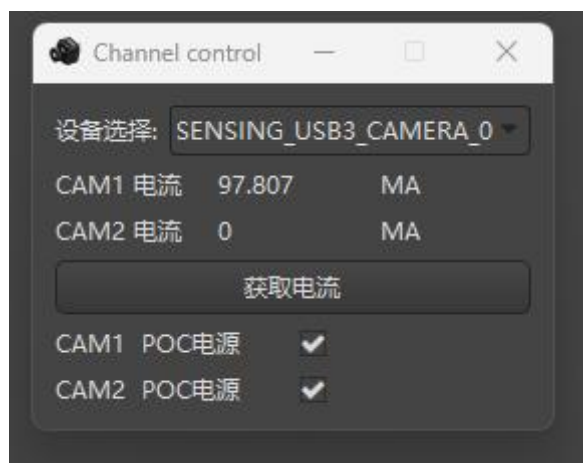


图 1-21

1.2.7 相机内参读取

如果相机进行了内参标定，可以通过上位机的读取内参功能将相机的内参读取出来。如图 1-22 在主界面左上角菜单栏中，选择工具->读取内参。在打开的界面中点击读取内参即可开始读取。如图 1-23 所示为读取的一个双目相机内参。



图 1-22

如果需要保存内参数据，点击导出参数，可以将读取的内参数据保存成 txt 文件。

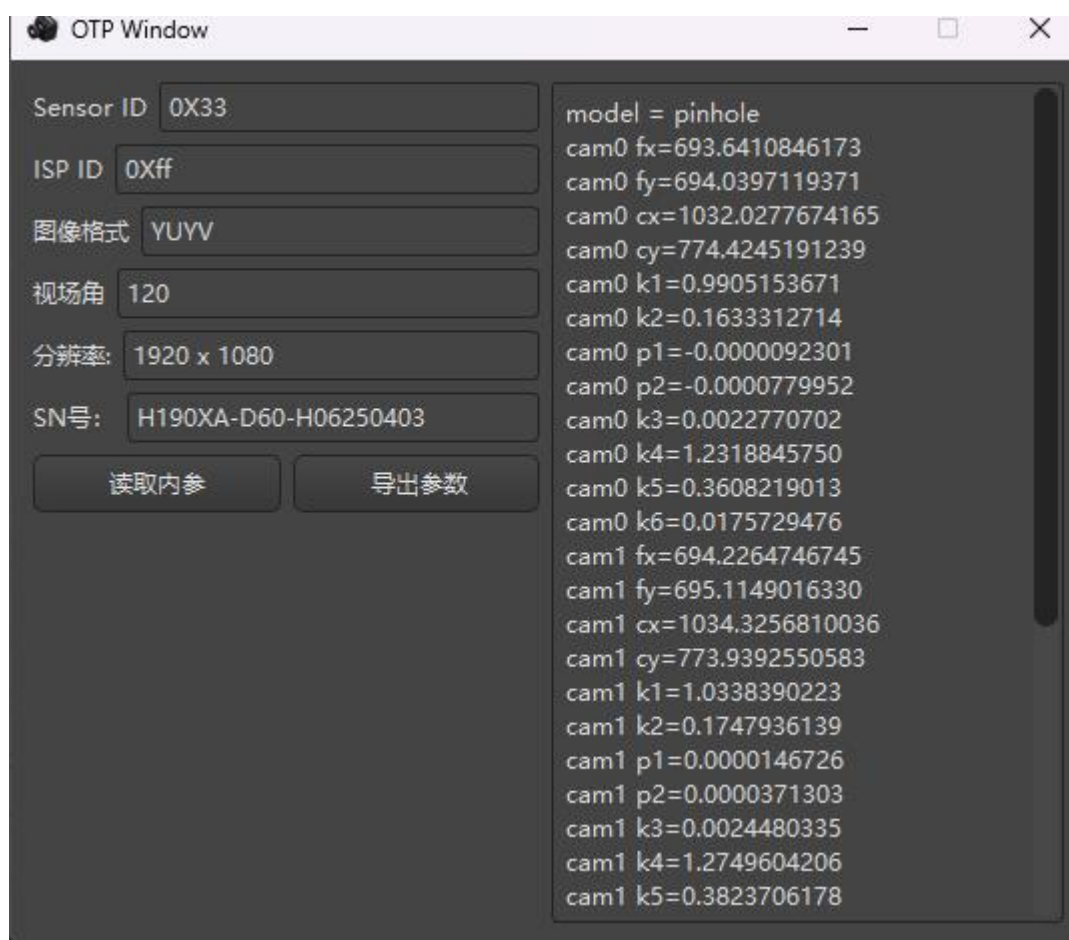


图 1-23

1.2.8 PANDA 固件更新

首次烧录需要安装驱动，驱动放在程序目录下 3rdparty/DriverInstallerCyUSB3 中。

步骤一：如图 1-24,框选的位置里面有一个按键，将这个按键（boot 键）按住，然后再将 USB3.0 的接口接上电脑（保证上电的时候按键是按下的状态）。



图 1-24

步骤二：打开上位机，在左上角，设置中，选择设备固件升级，如图 1-25 框选的位置所示。



图 1-25

步骤三：如果识别到了设备处于升级模式，会在弹出的窗口左侧列表中出现 Cypress FX3 USB BootLoader Device。如果没有出现，需要从步骤一开始重新操作。

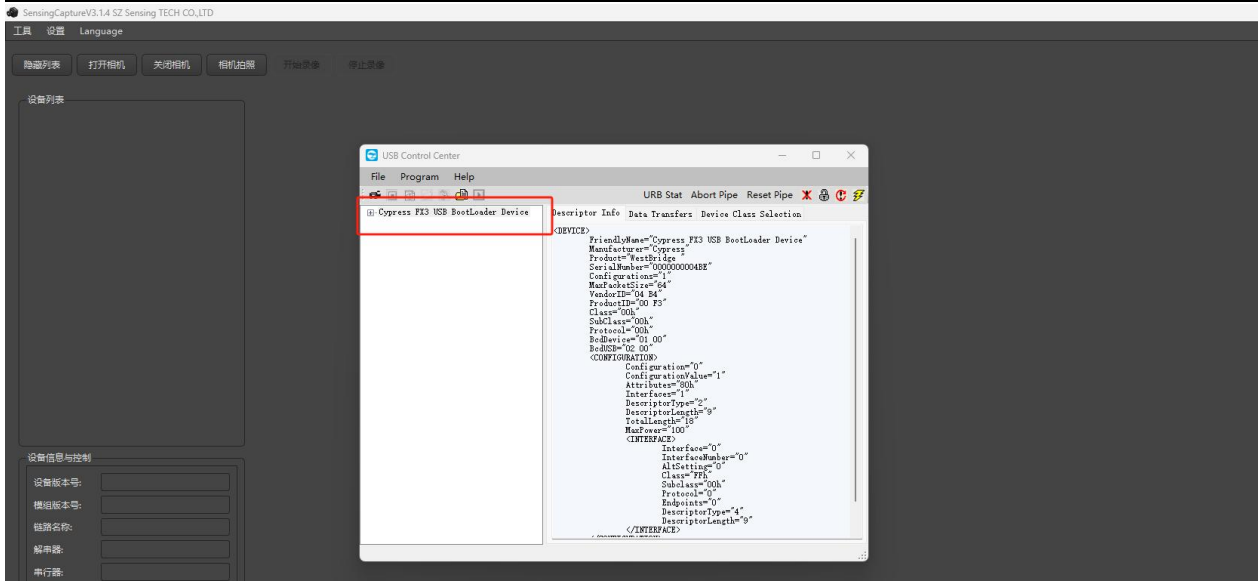


图 1-26

步骤四：如图 1-27 在升级界面中，选择 I2C EEPROM，选择过后会弹出选择路径的弹框。
选择需要升级的固件，点击打开。

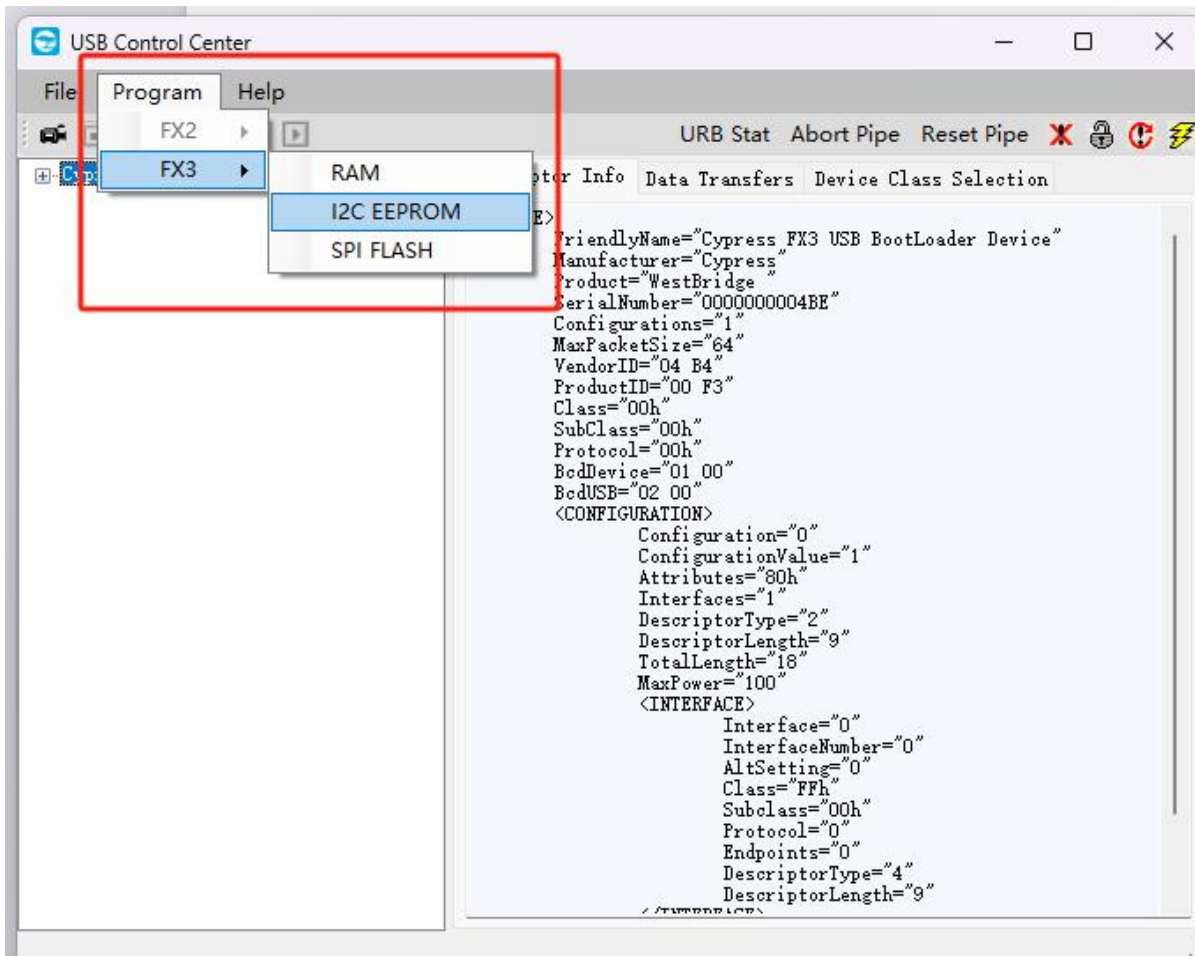


图 1-27

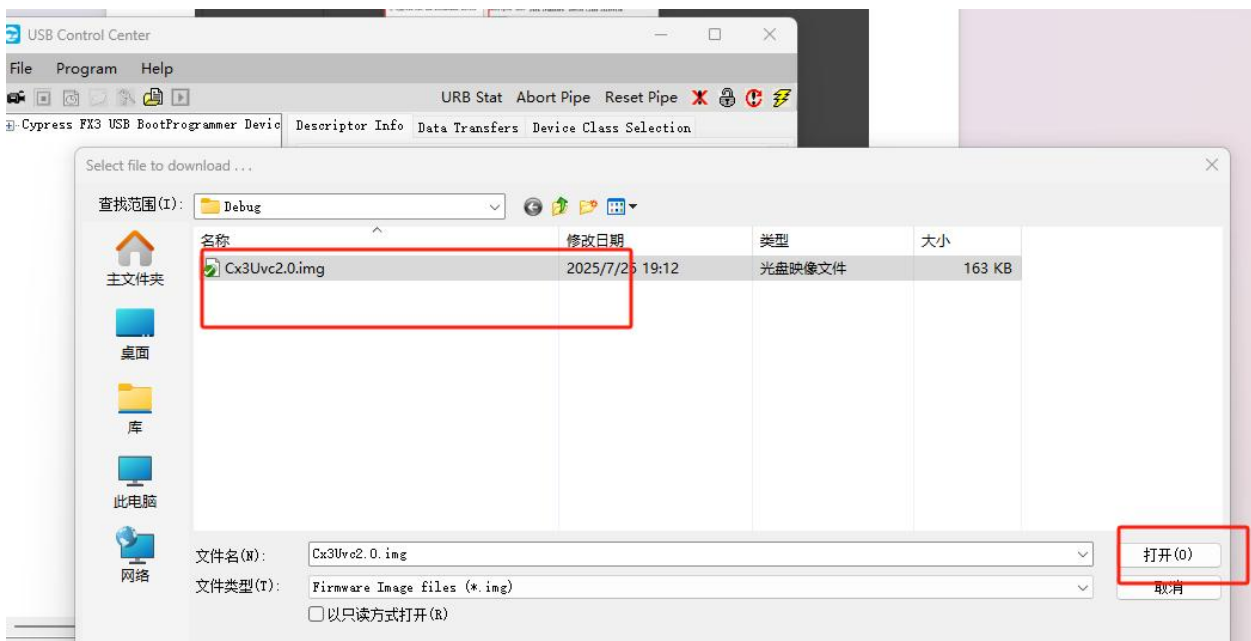


图 1-28

步骤五：选中后，如图 1-29 会在左下角显示升级中，等待出现如图 1-30 显示 Succeeded 表示升级成功。

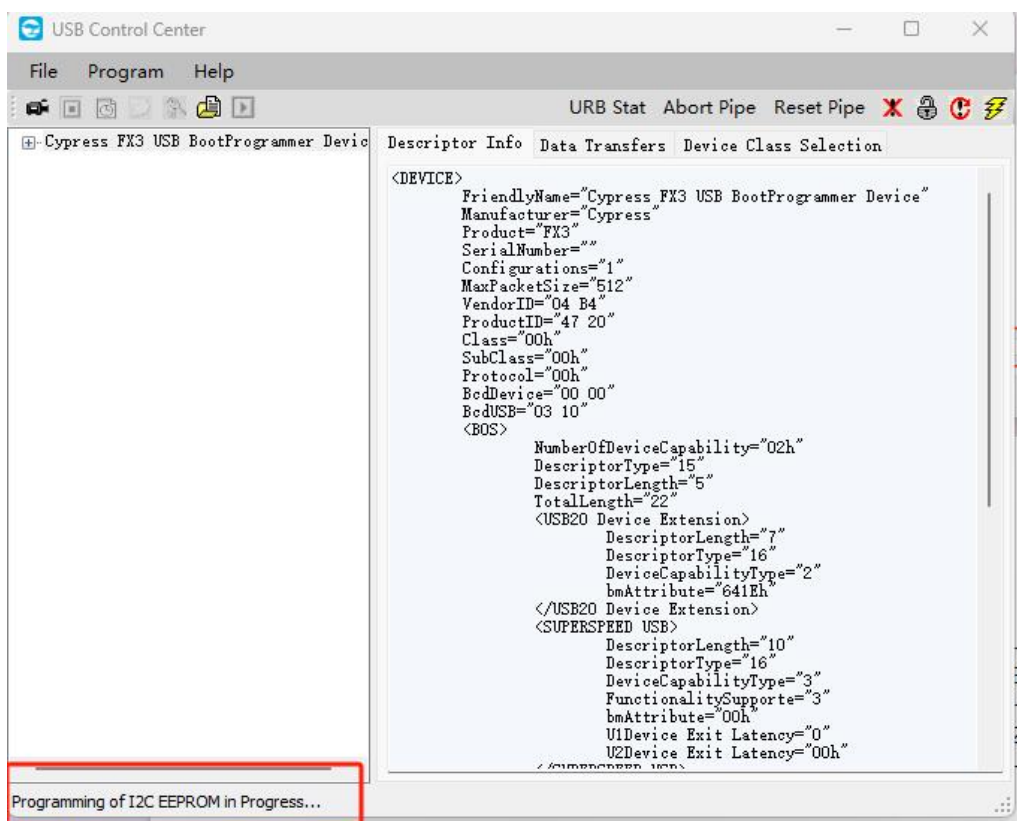


图 1-29

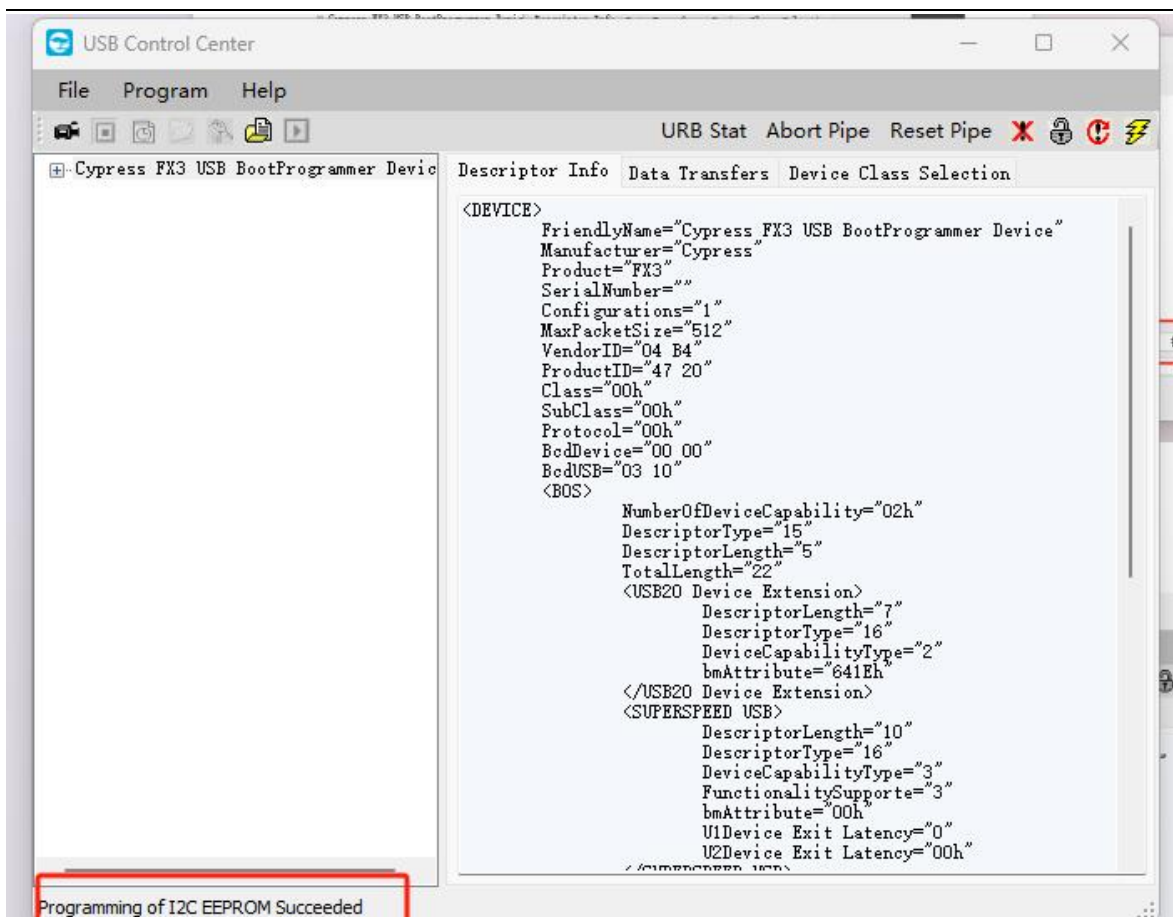


图 1-30

1.2.9 ISP Tuning

可以通过上位机 UI 界面简易调节图像效果，调节的效果不会保存在固件中，当重启模组后，会恢复到固件效果状态，但是可以将调节的参数导入和导出。如图 1-31，在主界面左上角菜单栏中 选择工具->ISP Tuning 打开效果调节界面。



图 1-31

打开界面后选择对应型号，如使用 SG3-ISX031C-G2F，打开后默认效果如图 1-32。

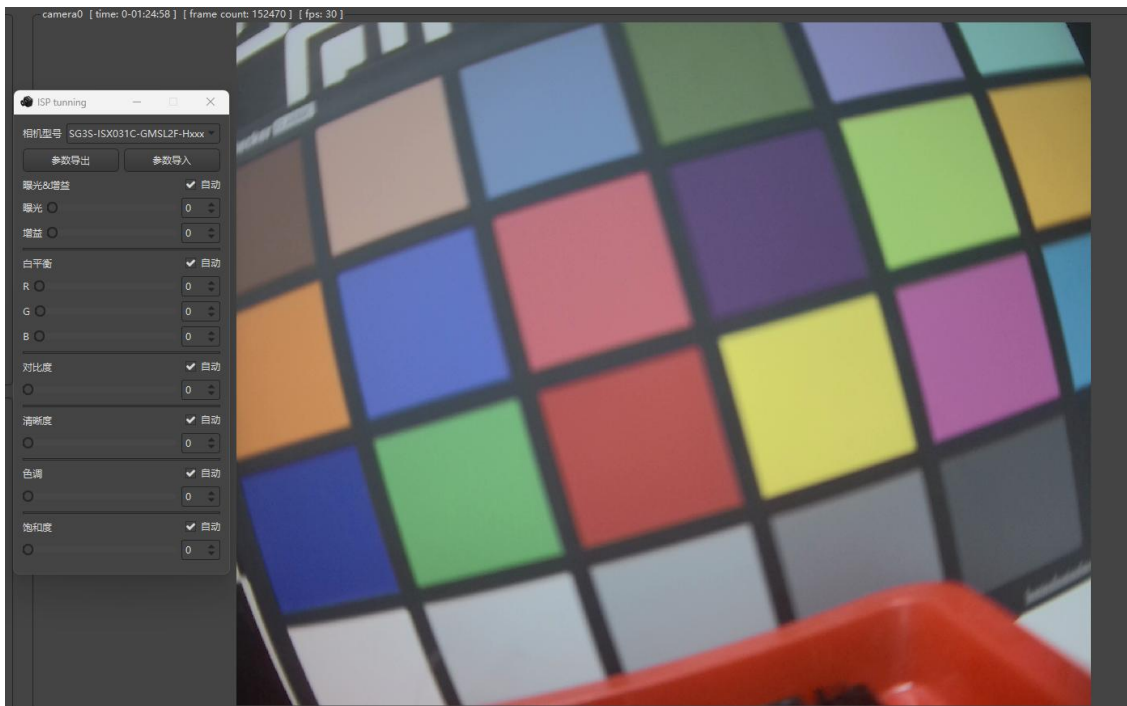


图 1-32

通过界面调节曝光，增益，白平衡等参数后，效果如图 1-33 所示。

点击按钮参数导出即可将当前参数保存成配置文件，当重新打开时，点击参数导入选择对应配置文件即可。

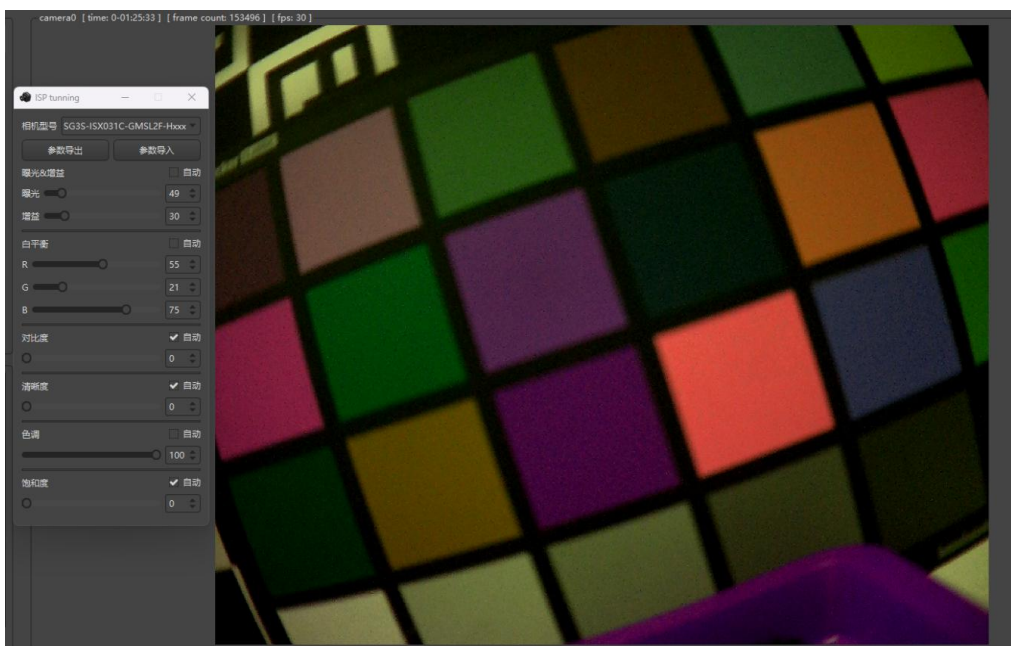


图 1-33

1.2.10 读取镜头温度

部分模组(RedFox-D3Gx,CMS)在镜头设置有传感器,可以通过读取该传感器获取镜头内侧当前温度。在主界面左上角 点击工具->读取温度, 打开读取温度界面如图 1-34 所示, 打开后点击按钮获取温度即可读取。

注: 此温度为镜头传感器处的温度, 非镜头表面温度。



图 1-34



图 1-35

二、Linux 使用说明

2.1 软件下载

支持的系统 (X86) : ubuntu18.04 , ubuntu20.04 , ubuntu22.04

下载指令:

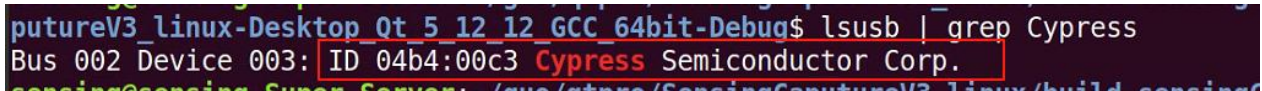
```
git clone -b Linux https://github.com/SENSING-Technology/SensingCaptureV3.git
```

2.2 使用方法

2.2.1 打开上位机

判断系统识别到 Panda, 使用指令 `lsusb | grep Cypress`

如果出现设备信息: ID 04b4:00c3 Cypress Semiconductor Corp. 则系统已经识别到设备。



```
sensingCaptureV3_linux-Desktop Qt 5 12 12 GCC 64bit-Debug$ lsusb | grep Cypress
Bus 002 Device 003: ID 04b4:00c3 Cypress Semiconductor Corp.
```

安装部分依赖库

```
sudo apt install libudev-dev
```

```
sudo apt-get install i2c-tools
```

进入软件文件夹路径, 右键在终端中打开。

输入指令: `sudo ./sensingCaputureV3_linux`

首次点亮需要添加权限

```
chmod 777 ./sensingCaputureV3_linux
```

打开界面如下

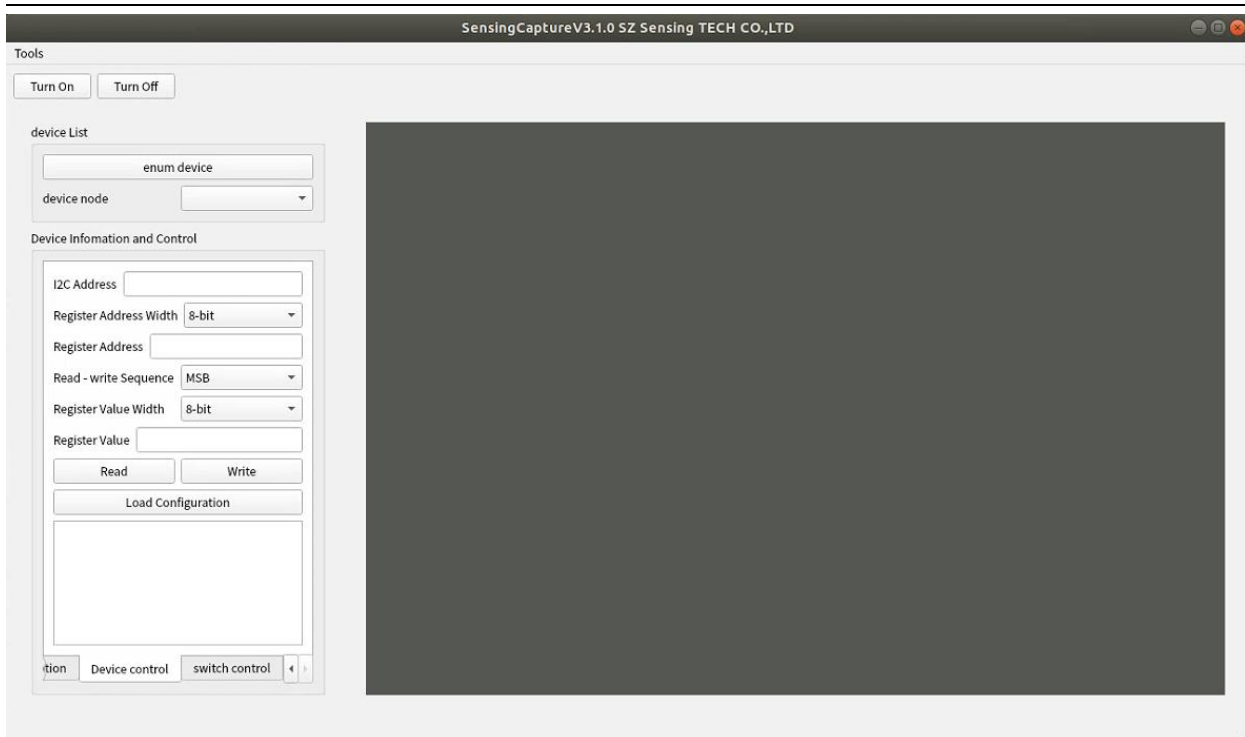


图 2-1

2.2.2 加载配置点亮相机

- 1、在 Device control 界面点击按钮 Load Configuration

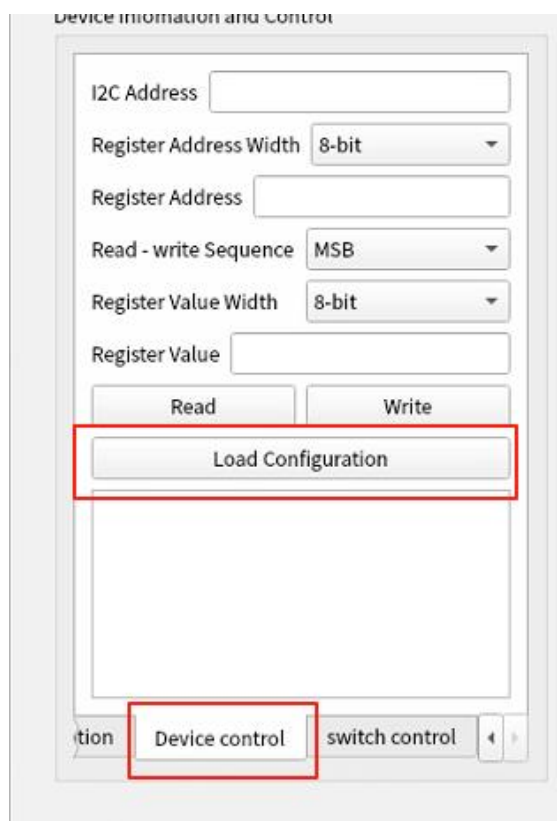


图 2-2

2、在弹出的窗口，先在模糊搜索（fuzzy search 后的输入框）输入型号的关键信息，如图 2-3。再打开下拉框选择对应的配置文件，选择好配置文件，点击 confirm 即可开始加载配置文件，如图 2-4。

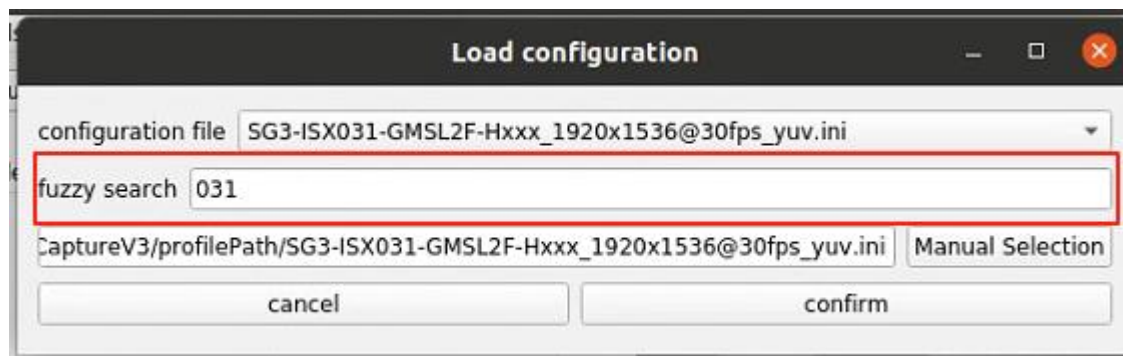


图 2-3



图 2-4

出现 reconnect 即加载配置文件成功，如图 2-5。

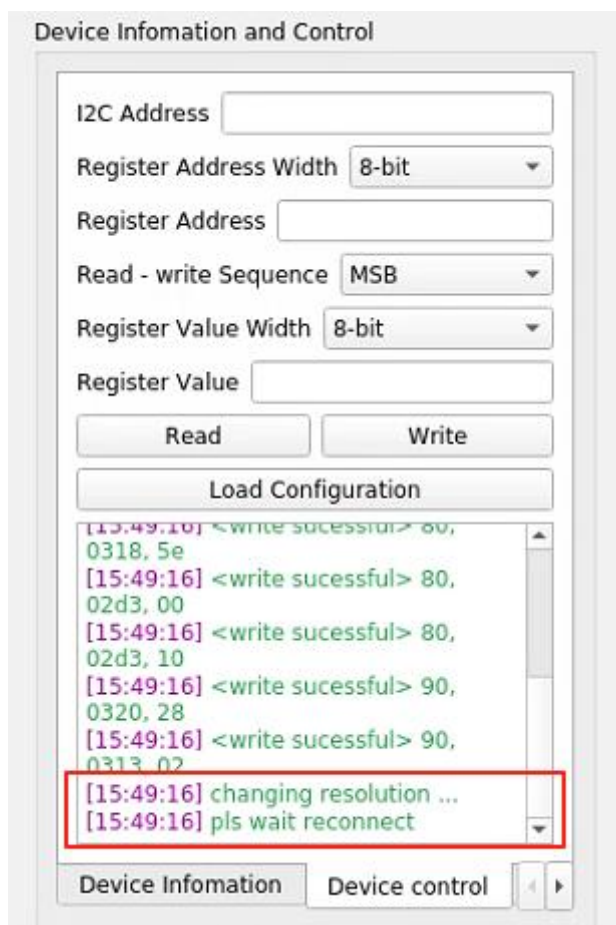


图 2-5

枚举设备节点：Panda 接上 Linux 系统，系统识别到会产生 2 个新的 video 节点，选择这两个中的第一个节点，打开相机时默认打开 video0 节点，如果有多个相机设备需要手动选择对应的节点。

使用命令 `v4l2-ctl -d /dev/videox --info` (x 需要替换为系统节点)

如果出现下图 card type 为: SENSING_USB3_CAMERA, 则该节点为 Panda 的设备节点。

```
guo@guo-TianYi510Pro-14ACN:~$ v4l2-ctl -d /dev/video0 --info
Driver Info:
  Driver name      : uvcvideo
  Card type        : SENSING USB3 CAMERA
  Bus info         : usb-0000:01:00.0-2
  Driver version   : 5.15.178
  Capabilities     : 0x84a00001
                    Video Capture
                    Metadata Capture
                    Streaming
                    Extended Pix Format
                    Device Capabilities
  Device Caps      : 0x04200001
                    Video Capture
                    Streaming
                    Extended Pix Format
guo@guo-TianYi510Pro-14ACN:~$
```

图 2-6

根据读取到的节点信息，选择对应的节点。

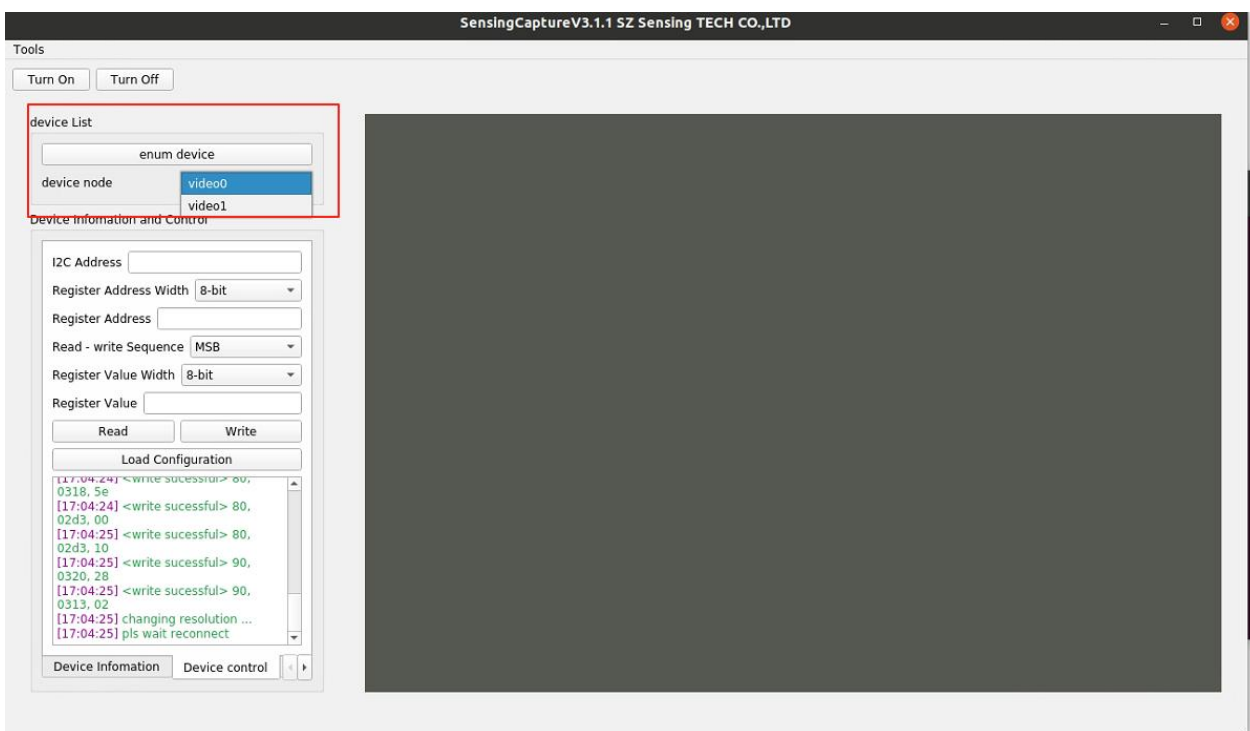


图 2-7

点击 Turn On 按钮打开相机，如图 2-8。

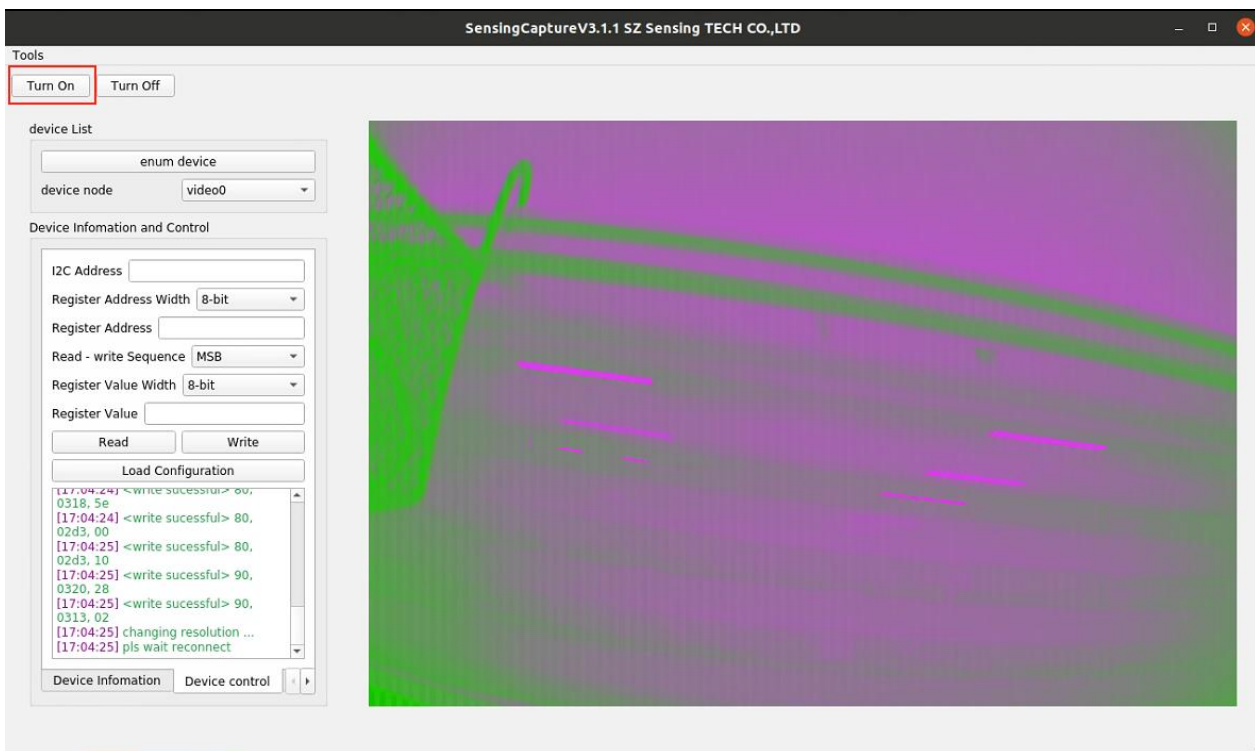


图 2-8

系统默认按照 YUYV 格式解析图像，如果图像是 UYVY 格式需要转换格式。在 Device information 界面选择 Image parsing Format 下拉框中的 UYVY，如下图 2-9，格式已经成功转换。

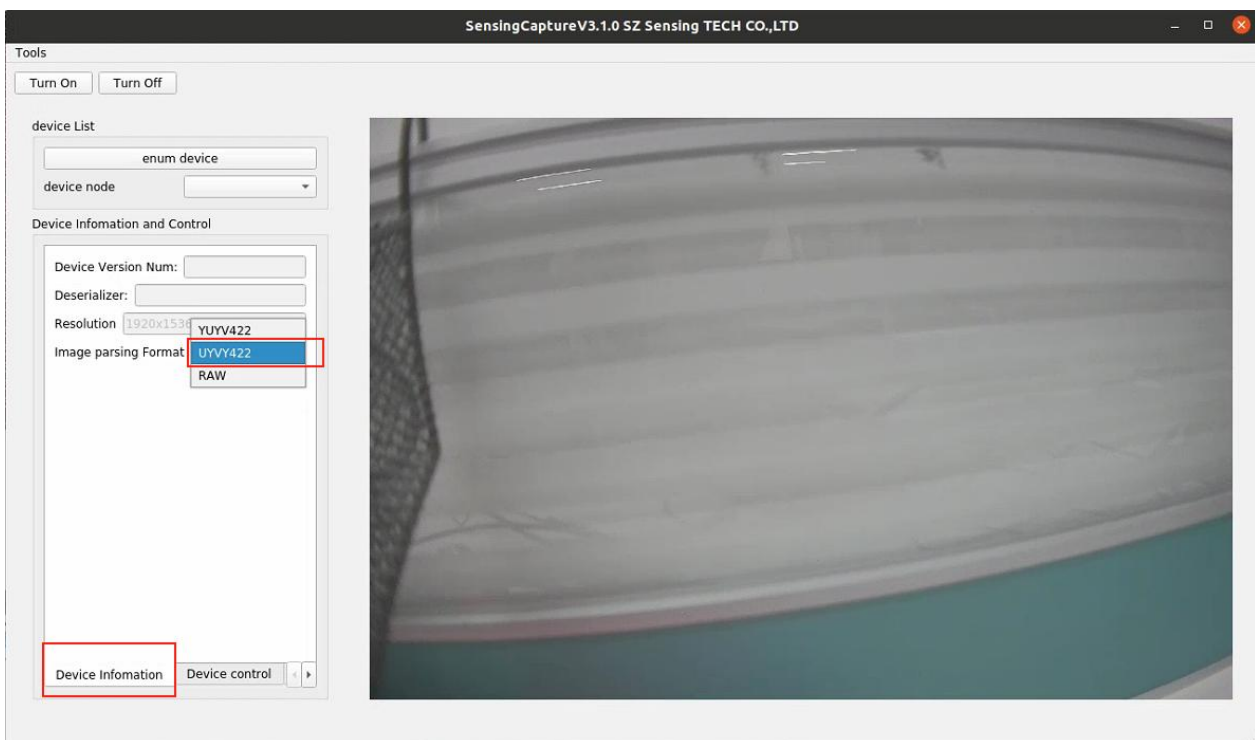


图 2-9

2.2.3 读写寄存器

上位机支持读写寄存器，方法如下

1、在 I2C Address 后的输入框中填入 I2C 地址,如下图读取 MAX9296 的工作模式, MAX9296 的 I2C 地址为 0x90, 在输入框中填写 90。

2、根据需求选择对应的寄存器宽度，目前支持 8bit 和 16bit 两种。9296 的寄存器宽度为 16bit，可以通过产品规格书，硬件配置页查看或 9296 的 DATASHEET。

3、在 Register Address 中填入需要读写的寄存器地址。如我需要读取 MAX9296 的工作模式，其对应的寄存器地址为 0x1，在 Register Address 后的输入框内填入 1。

4、选择 Read - write Sequence，选择高位在前还是低位在前，如果是写 8bit 数据可以忽略该选项；如果写 16bit 数据，并且需要低位数据在前可以选择 LSB，其他情况保持 MSB 即可。

5、选择写入的值宽度：值的宽度 8-bit 和 16-bit 可选。根据读写器件的手册选择，如示例这里需要读写 8-bit 选择 8-bit 即可。

6、输入写入的值：如果需要写入值，则在 Register Value 后的输入框内输入需要写入的值，如果是读取寄存器，则点击读取后会在该输入框内显示读取到的值。

7、在确认完写入或读取的信息，点击 Read 按钮可以执行读取操作，点击 Write 按钮可以执行写入按钮。

如下图，如果读取数据成功，则在显示的框内显示 <read sucessful>

如果写入数据成功，则在显示框内显示<write sucessful>

如果读取数据异常，则在显示的框内显示 <read fail>

如果写入数据异常，则在显示框内显示<write fail>

注意：若在出图过程中读写寄存器会中断出图，需要完成操作后重新操作点亮相机步骤。

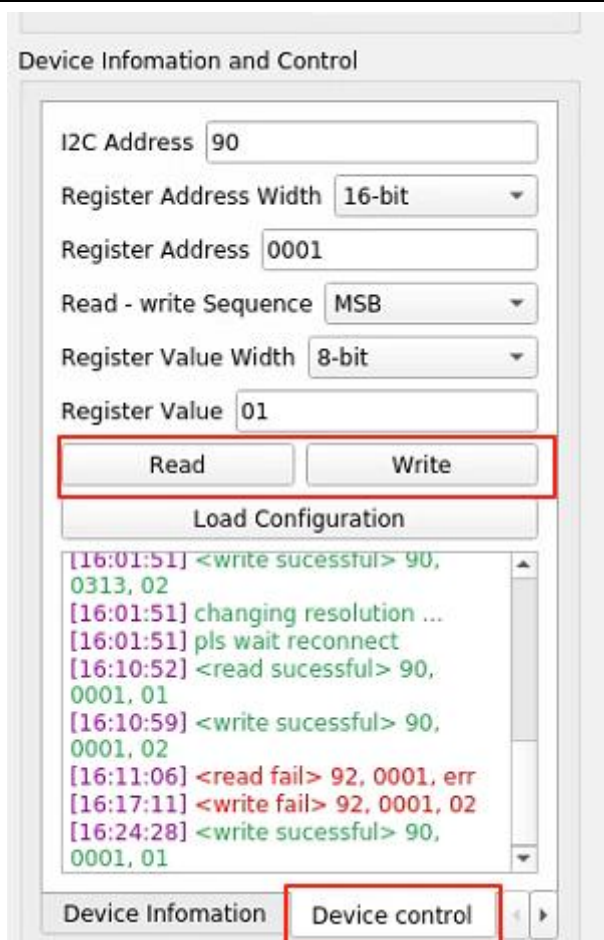


图 2-10

2.2.4 开关控制

在 switch control 界面可以读取电流，和控制相机的电源开关。

当将相机接在 Panda 上，通过读取电流的方式可以估算当前相机的功耗。单击 Obtain current 按钮获取电流，可以查看当前模组的电流。如图所示读取到的电流为 104.32MA。

Channel 1 后的可选框用于控制 CAM1 端口的 POC 电源，若取消勾选则会断开 CAM1 的 POC 电源。

Channel 2 后的可选框用于控制 CAM2 端口的 POC 电源，若取消勾选则会断开 CAM2 的 POC 电源。

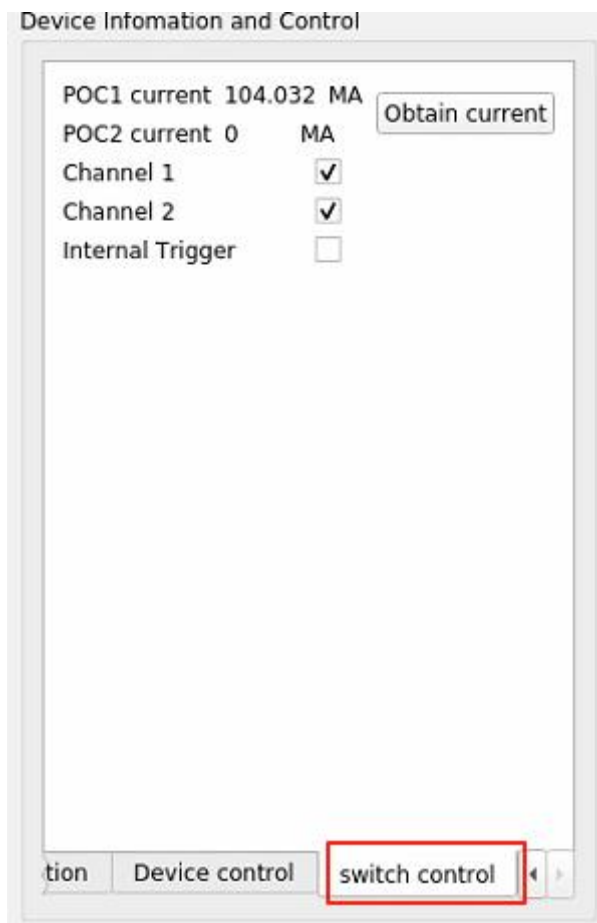


图 2-11

2.3 Linux 系统脚本点亮方式

在 PANDA 盒子默认固件只能点亮分辨率为 1920*1080 的模组，点亮其他分辨率的模组需要先加载配置，配置文件和 windows 的配置文件一致。

首先获取 Linux 的加载配置的程序

下载链接：<https://github.com/SENSING-Technology/PandaCtrl>

如图 2-12，在程序包里面应该包含 PandaCtrl 文件夹，和 libusb 的源码文件，在程序文件里面默认是在 ubuntu20.04 系统编译的 libusb 库，如果是在其他系统上使用需要重新编译 libusb 库，替换生成的库文件，再编译加载配置文件的程序。

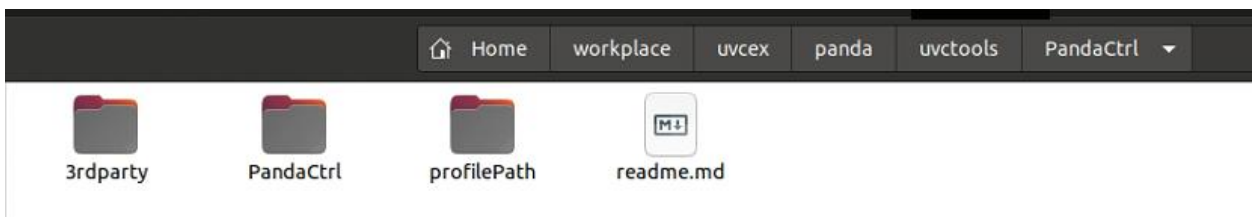


图 2-12

打开 PandaCtrl 文件夹，运行里面的 makefile 文件。

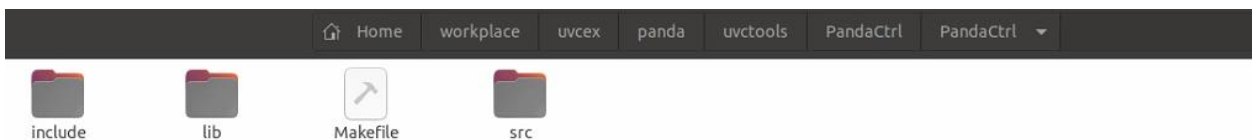


图 2-13

执行 makefile 文件后，如果编译成功会多出一个可执行程序 PandaCtrl 文件。

注意：需要安装 gcc 和 make 工具。

部分系统需要按照一些依赖库，如 ubuntu20.04 需要执行下列命令

`sudo apt install libudev-dev`

`sudo apt-get install i2c-tools`

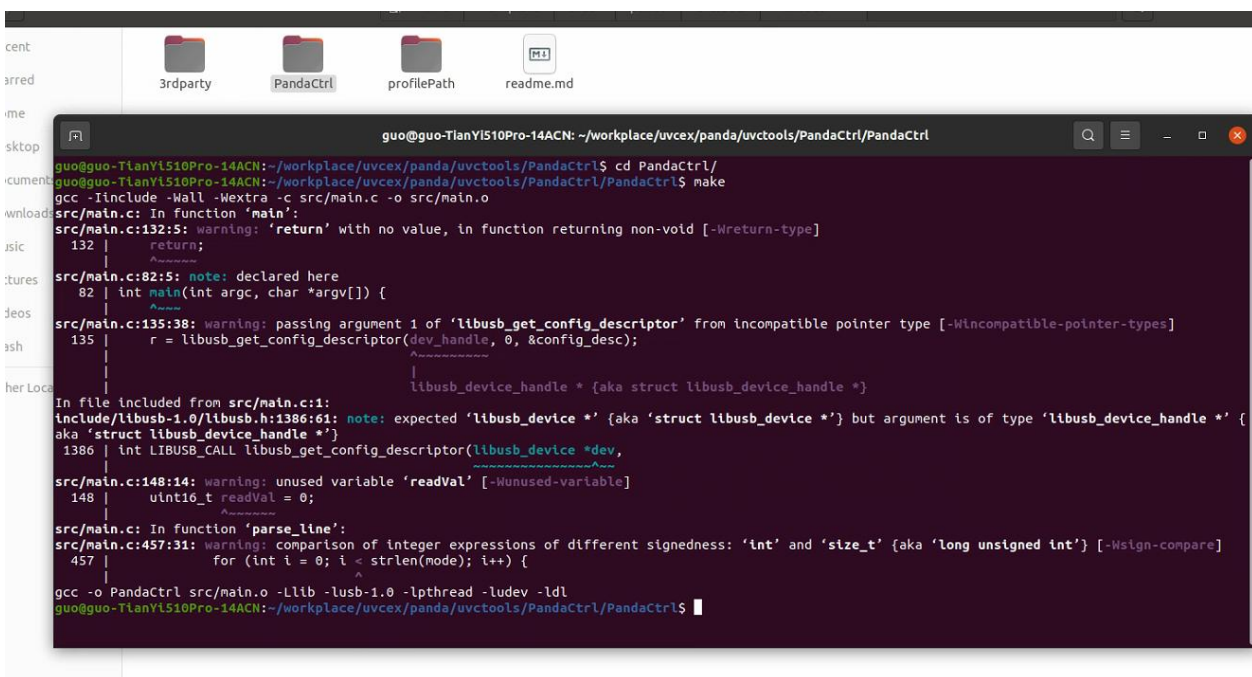


图 2-14

运行的格式为：sudo ./PandaCtrl [配置文件的绝对路径]

如图 44 所示，运行后有提示 Device opened successfully 表示正常加载配置文件。

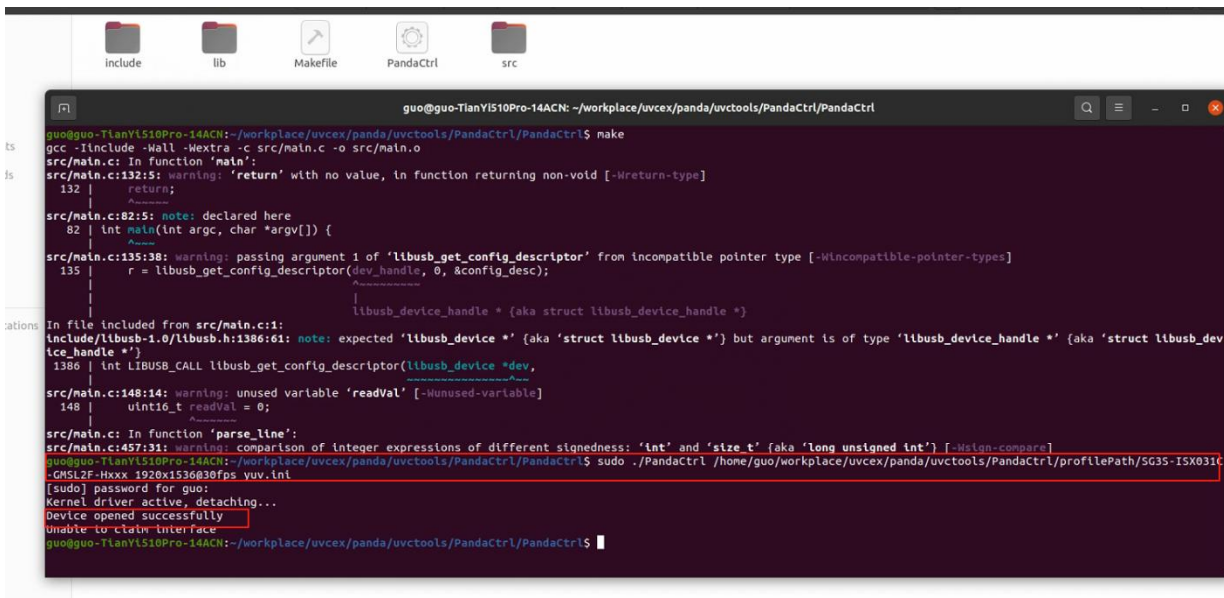


图 2-15

加载完成后打开系统的相机，若相机能正常打开，表示配置成功。如果打开相机能出图但是色域不对，可以参考附录中的 UYVY 和 YUYV 格式的相互转换。

如图 2-16，如果需要重新编译 libusb 的库，在 libusb-1.0.26 中编译后，用 libusb/.lib/libusb-1.0.a 文件替换掉 PandaCtrl/lib/libusb-1.0.a。

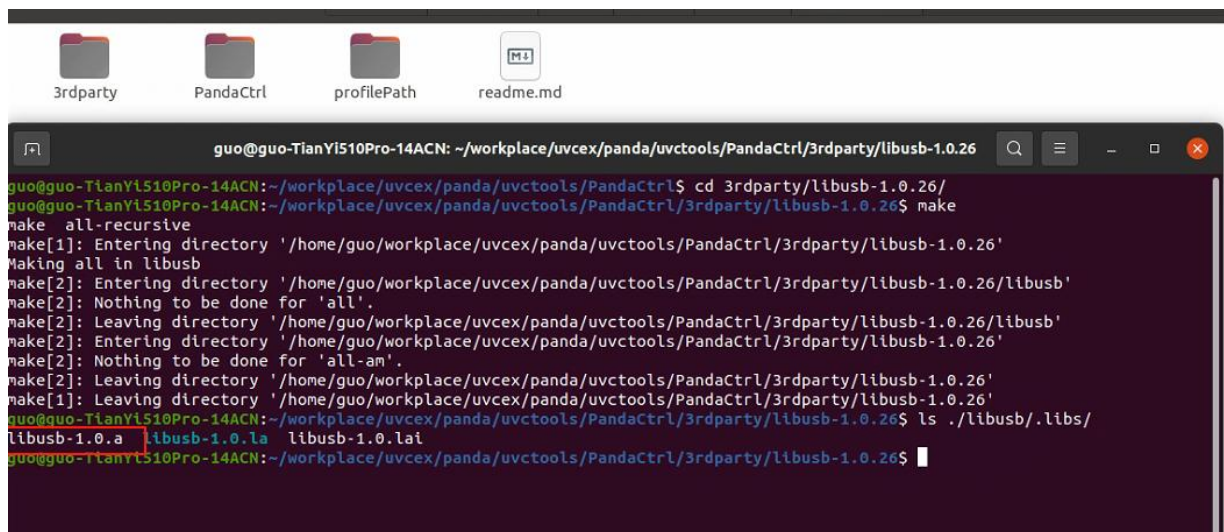


图 2-16

三、版本记录

版本	描述	日期
Ver 1	首次发行	2025/06
Ver 2	1、增加 Panda PG2 固件升版内容 2、增加 Linux 系统点亮的内容 3、增加适配的森云相机型号 4、增加附录 UYVY 和 YUYV 的格式转换	2025/08
Ver 3	1、完善在 Linux 系统上使用的章节 2、删除适配的森云相机型号	2025.09
Ver 4	完善 Windows 系统下的使用功能 1、设置触发 2、ISP Tuning 3、读取镜头温度	2025.10

附录

一、UYVY 转 YUYV python 示例

1.1 实现思路

理解 YUV422 UYVY 和 YUYV 格式：

UYVY 格式：每两个相邻的像素共享一组 UV 分量，排列顺序是 U0 Y0 V0 Y1。

YUYV 格式：同样每两个相邻的像素共享一组 UV 分量，排列顺序是 Y0 U0 Y1 V0。

数据转换：要将 UYVY 转换为 YUYV，只需要对数据进行重新排列即可。

1.2 python 示例代码仅供参考：

```
import cv2

import numpy as np

def uyvy_to_yuyv(uyvy_data, width, height):
    # 将输入的 UYVY 数据转换为 numpy 数组
    uyvy_array = np.frombuffer(uyvy_data, dtype=np.uint8)
    # 调整数组形状以匹配图像尺寸
    uyvy_array = uyvy_array.reshape((height, width * 2))
    # 创建一个空的 YUYV 数组
    yuyv_array = np.empty_like(uyvy_array)
    # 对每一行进行处理
    for i in range(height):
        for j in range(0, width * 2, 4):
            # 重新排列数据，从 UYVY 转换为 YUYV
            yuyv_array[i, j] = uyvy_array[i, j + 1]
            yuyv_array[i, j + 1] = uyvy_array[i, j]
```

```

        yuyv_array[i, j + 2] = uyvy_array[i, j + 3]

        yuyv_array[i, j + 3] = uyvy_array[i, j + 2]

    # 将 YUYV 数组转换为字节数据

    yuyv_data = yuyv_array.tobytes()

    return yuyv_data

# 示例用法

width = 640

height = 480

# 生成随机的 UYVY 数据作为示例

uyvy_data = np.random.randint(0, 256, width * height * 2,
dtype=np.uint8).tobytes()

# 调用转换函数

yuyv_data = uyvy_to_yuyv(uyvy_data, width, height)

# 将 YUYV 数据转换为 OpenCV 图像

yuyv_array = np.frombuffer(yuyv_data, dtype=np.uint8).reshape((height,
width * 2))

# 将 YUYV 格式的图像转换为 BGR 格式，以便在 OpenCV 中显示

bgr_image = cv2.cvtColor(yuyv_array, cv2.COLOR_YUV2BGR_YUYV)

# 显示图像

cv2.imshow('YUYV Image', bgr_image)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

1.3 代码解释：

(1) 定义转换函数 `uyvy_to_yuyv`: 该函数接受 UYVY 数据、图像宽度和高度作为输入, 将 UYVY 数据转换为 YUYV 数据。

(2) 数据转换: 通过双重循环遍历 UYVY 数据的每一行和每四个字节, 将数据按照 YUYV 的排列顺序重新排列。

(3) 示例用法: 生成随机的 UYVY 数据作为示例, 调用转换函数将其转换为 YUYV 数据。

(4) 图像显示: 将 YUYV 数据转换为 OpenCV 图像, 并将其从 YUYV 格式转换为 BGR 格式, 最后显示图像。

1.4 注意事项

(1) 确保输入的 UYVY 数据的长度与图像的宽度和高度相匹配。

(2) 代码中的随机 UYVY 数据仅用于示例, 实际应用中需要使用真实的 UYVY 数据。

二、YUYV 转 UYVY python 示例

2.1 实现思路

(1) YUYV 和 UYVY 格式特点:

YUYV 格式: 每两个相邻像素共享一组 UV 分量, 数据排列为 Y0 U0 Y1 V0 。

UYVY 格式: 同样是每两个相邻像素共享一组 UV 分量, 不过排列顺序是 U0 Y0 V0 Y1 。

(2) 转换方法: 本质上就是对数据进行重新排列, 把 YUYV 格式里的数据按照 UYVY 的顺序重新组合。

2.2 python 示例代码-仅供参考

```
import cv2

import numpy as np

def yuyv_to_uyvy(yuyv_data, width, height):
```

```
# 将输入的 YUYV 数据转为 numpy 数组

yuyv_array = np.frombuffer(yuyv_data, dtype=np.uint8)

# 调整数组形状以匹配图像尺寸

yuyv_array = yuyv_array.reshape((height, width * 2))

# 创建一个空的 UYVY 数组

uyvy_array = np.empty_like(yuyv_array)

# 逐行处理数据

for i in range(height):

    for j in range(0, width * 2, 4):

        # 按照 UYVY 的顺序重新排列数据

        uyvy_array[i, j] = yuyv_array[i, j + 1]

        uyvy_array[i, j + 1] = yuyv_array[i, j]

        uyvy_array[i, j + 2] = yuyv_array[i, j + 3]

        uyvy_array[i, j + 3] = yuyv_array[i, j + 2]

# 将 UYVY 数组转换为字节数据

uyvy_data = uyvy_array.tobytes()

return uyvy_data

# 示例使用

width = 640

height = 480

# 生成随机的 YUYV 数据作为示例

yuyv_data = np.random.randint(0, 256, width * height * 2,

dtype=np.uint8).tobytes()
```

```
# 调用转换函数

uyvy_data = yuyv_to_uyvy(yuyv_data, width, height)

# 将 UYVY 数据转换为 OpenCV 图像

uyvy_array = np.frombuffer(uyvy_data, dtype=np.uint8).reshape((height,
width * 2))

# 将 UYVY 格式的图像转换为 BGR 格式，以便在 OpenCV 中显示

bgr_image = cv2.cvtColor(uyvy_array, cv2.COLOR_YUV2BGR_UYVY)

# 显示图像

cv2.imshow('UYVY Image', bgr_image)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

2.3 代码解释

2.3.1 yuyv_to_uyvy 函数：该函数接收 YUYV 数据、图像宽度和高度作为输入。

- (1) 先把输入的 YUYV 数据转换为 numpy 数组，再调整其形状以匹配图像尺寸。
- (2) 创建一个与 yuyv_array 相同大小的空 uyvy_array，用于存储转换后的数据。
- (3) 通过两层嵌套循环逐行处理数据，将 YUYV 数据按照 UYVY 的排列规则重新赋值给 uyvy_array。

- (4) 最后将转换好的 uyvy_array 转换为字节数据并返回。

2.3.2 示例使用部分：

- (1) 设定图像的宽度和高度，这里分别为 640 和 480。
- (2) 生成随机的 YUYV 数据作为示例输入。
- (3) 调用 yuyv_to_uyvy 函数进行格式转换。
- (4) 将转换后的 UYVY 数据转换为 numpy 数组，并通过 cv2.cvtColor 函数将其从

UYVY 格式转换为 BGR 格式，这样就能使用 cv2.imshow 函数显示图像了。

(5) 最后使用 cv2.waitKey(0)等待用户按键，使用 cv2.destroyAllWindows()关闭所有打开的窗口。

2.4 注意事项

- (1) 要保证输入的 YUYV 数据长度和所设定的图像宽度、高度相匹配。
- (2) 代码里生成的随机 YUYV 数据仅作示例，实际应用时需要使用真实的 YUYV 数据。